Earners AI and ML by Aditya Jain sir - Lecture 19

Summary Notes

OOP using Python

```
# class --> Blueprint
# Object --> spcific instance of that class
```

```
name = "Raj"
age = 20
city  = "Mumbai"
```

```
name1 = "Raj"
age1 = 20
city1  = "Mumbai"
```

```
name2 = "Rahul"
age2 = 25
city2  = "Delhi"
```

```
name3 = "Ayushi"
age3 = 21
city3  = "Pune"
```

```
# Before manufacturing the car, the company would finalize the template.

# Features of the car that should be present in its blueprint:
# 1. Brand
# 2. Color
# 3. Price

# this is the blueprint and the actual car --> object/intance of a class
```

```
# eg2: House
# features:
# 1. bhk
# 2. sqft area
# 3. location
```

class creation

```
class Student:
```

```
  File "/tmp/ipykernel_572/777707742.py", line 2

    ^
SyntaxError: incomplete input
```

Next steps: Explain error

```
class Student:
  pass #placeholder --> do nothing
```

```
class Student:
  name = "Raj"
  age = 21
  city = "Mumbai"
```

```
s1 = Student()
```

```
print(s1.name)
```

```
Raj
```

```
print(s1.age)
```

```
21
```

```
print(s1.city)
```

```
Mumbai
```

```
s2 = Student()
```

```
print(s2.name)
```

```
Raj
```

```
print(s2.age)
```

```
21
```

```
print(s2.city)
```

```
Mumbai
```

```
class Student:
  name
  age
  city
```

```
s1 = Student()
```

```
s1.name = "Raj"
s1.age = 21
s1.city = "Mumbai"
```

```
s2 = Student()
s2.name = "Ayushi"
s2.age = 20
s2.city = "Pune"
```

```
print(s1.name, s2.name)
```

```
Raj Ayushi
```

```
#this repetitive problem can be solved using __init__()
```

```
class Student:

  def __init__():
    print("Student object has got created!")
```

```
s1 = Student()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_572/391622993.py in <cell line: 0>()
----> 1 s1 = Student()

TypeError: Student.__init__() takes 0 positional arguments but 1 was given
```

Next steps: Explain error

```
class Student:

  def __init__(self):
    print("Student object has got created!")
```

```
s1 = Student()
```

```
Student object has got created!
```

```
class Student:

  def __init__(self, name):
    self.name = name
```

```
s1 = Student("Raj")
```

```
s1.name
```

```
'Raj'
```

```
s2 = Student("Rahul")
print(s2.name)
```

```
Rahul
```

```
s3 = Student("Ayushi")
print(s3.name)
```

```
Ayushi
```

```
class Student:

  def __init__(self, name, age, city):
    self.name = name
    self.age = age
    self.city = city
```

```
s1 = Student("Raj", 21, "Mumbai")
s2 = Student("Rahul", 20, "Pune")
s3 = Student("Ayushi", 22, "Delhi")
```

```
print(s1.name, s2.name, s3.name)
print(s1.age, s2.age, s3.age)
print(s1.city, s2.city, s3.city)
```

```
Raj Rahul Ayushi
21 20 22
Mumbai Pune Delhi
```

```
class Student:

  def __init__(self, name, age, city):
    self.name = name
    self.age = age
    self.city = city

  def display_profile(self):
    print(self.name, self.age, self.city)
```

```
s1 = Student("Raj", 21, "Mumbai")
s2 = Student("Rahul", 20, "Pune")
s3 = Student("Ayushi", 22, "Delhi")
```

```
s1.display_profile()
```

```
Raj 21 Mumbai
```

```
s2.display_profile()
```

```
Rahul 20 Pune
```

Mini-Case Study

```
A bank wants to build a simple system to manage customer accounts using Object-Oriented Programming (OOP).

Create a class named BankAccount with the following requirements:

Requirements
The class should have a constructor (__init__) that accepts:
Account holder's name (holder)
Initial account balance (balance)

Create a method named deposit(amount) that adds the given amount to the account balance.

Create a method named withdraw(amount) that:
Deducts the given amount from the balance if sufficient funds are available.

Otherwise, displays the message:
Insufficient Balance

Create a method named show_balance() that displays the current account balance.
```

```
class BankAccount:

  def __init__(self, holder, balance):
    self.holder = holder
    self.balance = balance

  def deposit(self, amount):
    self.balance += amount

  def withdraw(self, amount):
    if amount > self.balance:
      print("Insufficient Balance")
    else:
      self.balance -= amount

  def show_balance(self):
    print(self.balance)
```

```
bank_acc1 = BankAccount("Raj", 100000)
```

```
bank_acc1.holder
```

```
'Raj'
```

```
bank_acc1.balance
```

```
100000
```

```
bank_acc1.show_balance()
```

```
100000
```

```
Start coding or generate with AI.
```

```
bank_acc1.deposit(50000)
```

```
bank_acc1.show_balance()
```

```
150000
```

```
bank_acc1.withdraw(20000)
```

```
bank_acc1.show_balance()
```

```
130000
```

```
bank_acc1.withdraw(200000)
```

```
Insufficient Balance
```

QUiZ

```
What is the output?

class Student:
    pass

s = Student()

print(type(s))

A.

<class 'Student'>

B.

<class '__main__.Student'>

C.

Student

D. Error
```

```
class Student:
    pass

s = Student()

print(type(s))
```

```
<class '__main__.Student'>
```

```
How do you create an object of the class Car?

A.

Car

B.

Car[]

C.

Car()

D.

new Car()
```

```
What is the output?

class Student:
    pass

s1 = Student()
s2 = Student()

print(s1 == s2)

A. True
```

```
B. False

C. None

D. Error
```

```
class Student:
    pass

s1 = Student()
s2 = Student()

print(s1 == s2)
```

```
False
```

```
What is the purpose of the __init__() method?

A. Deletes an object

B. Initializes an object when it is created

C. Prints an object

D. Stops the program


Ans: B
```

```
class Student:

    def __init__(self):
        print("Student Created")

s = Student()
```

```
Student Created
```

```
class Student:

    def __init__(self, name):
        self.name = name

s = Student("Rahul")

print(s.name)
```

```
Rahul
```

```
What is self?

A. Name of the class

B. Current object

C. Constructor

D. Function name
```

```
# What is the output?

class Car:

    def __init__(self, brand):
        self.brand = brand

car = Car("BMW")

print(car.brand)
```

```
BMW
```

```
class Student:

    def __init__(self, name):
        self.name = name

s1 = Student("Rahul")
s2 = Student("Neha")

print(s2.name)
```

```
Neha
```

```
class Employee:

    def __init__(self, salary):
        self.salary = salary

emp = Employee(50000)

emp.salary = 60000

print(emp.salary)
```

```
60000
```

HW

```
class Mobile:

    def __init__(self, brand):
        self.brand = brand

m = Mobile("Samsung")

print(hasattr(m, "brand"))
```

```
What is the output?

class Student:

    def show(self):
        print("Python")

s = Student()

print(type(s))

A.

<class '__main__.Student'>

B.

Student

C.

Python

D. Error
```

```
What is the output?

class Book:

    def __init__(self, title):
        self.title = title

b = Book("Python Basics")

print(len(b.title))
```

```
What is the output?

class Rectangle:

    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width

r = Rectangle(5, 4)

print(r.area())
```

```
What is the output?

class Circle:

    def __init__(self, radius):
        self.radius = radius

c1 = Circle(5)
c2 = Circle(10)

print(c1.radius + c2.radius)
```

```
What is the output?

class Laptop:

    def __init__(self, brand):
        self.brand = brand

    def display(self):
        print(self.brand)

l = Laptop("Dell")

l.display()
l.display()
```

```
class Bank:

    def __init__(self, balance):
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

account = Bank(1000)

account.deposit(500)

print(account.balance)
```

```
class Student:

    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def result(self):
        if self.marks >= 40:
            print("Pass")
        else:
            print("Fail")

s = Student("Rahul", 35)

s.result()
```

```
class Student:
```

```
    def __init__(self, name):
        self.name = name

    def change_name(self, new_name):
        self.name = new_name

s = Student("Rahul")

s.change_name("Amit")

print(s.name)
```

```
class Counter:

    def __init__(self):
        self.count = 0

    def increase(self):
        self.count += 1

c = Counter()

for i in range(5):
    c.increase()

print(c.count)
```